Звіт з розрахунково-графічної роботи

Студент: Степаненко Денис, група ІМ-051 **Номер у списку групи (Ж):** 13 **Варіант:** 13 — програма списку студентів групи з розрахунками успішності

---

1. Завдання

Розробити програму для ведення списку студентів групи з розрахунками успішності: додавання/редагування/видалення, табличне відображення, сортування, статистика групи, збереження та завантаження файлу.

2. Аналіз та проектування

Програма розділена на рівні:

- **модель** — класи `Student` та `StudentGroup` (чисте ООП, без прив'язки до UI);
- **інтерфейс** — головне вікно з `ListView`, меню, модальні діалоги Add/Edit та статистики.

Клас `Student` зберігає дані студента (ПІБ, № залікової, 5 оцінок) і реалізує похідні розрахунки: `avg()`, `hasDebt()`, `isExcellent()`.

Клас `StudentGroup` інкапсулює `vector<Student>`: операції add/update/remove, сортування за балом/ПІБ, статистику групи, збереження/завантаження CSV.

UI-модуль не знає про внутрішню структуру даних — працює тільки через публічні методи контейнера.

3. Ключові рішення

Рішення	Обґрунтування
`vector<Student>` усередині класу	інкапсуляція — UI не торкається контейнера напряму
`wstring`/`wifstream` + UTF-8	коректна українська кирилиця у файлі
`ListView` у режимі `LVS_REPORT`	стандартна таблиця Windows, сортування за кліком неможливе — сортування через меню
Модальний діалог Add/Edit	один `DlgProc` для обох режимів, результат — через `DialogBoxParam`
Формат CSV (`;`)	сумісний з Excel, читається людиною

4. Лістинги

student.h

```
#pragma once
#include <string>
```

```
class Student
{
public:
    static const int GRADE_COUNT = 5;
private:
    std::wstring name;
    std::wstring bookNo;
    int grades[GRADE_COUNT];
public:
    Student();
    Student(const std::wstring& name, const std::wstring& bookNo);
    const std::wstring& getName()   const { return name; }
    const std::wstring& getBookNo() const { return bookNo; }
    int getGrade(int i) const;
    void setName(const std::wstring& v)   { name = v; }
    void setBookNo(const std::wstring& v) { bookNo = v; }
    void setGrade(int i, int v);
    double avg()         const;
    bool   hasDebt()     const;
    bool   isExcellent() const;
    std::wstring toCSV() const;
    bool fromCSV(const std::wstring& line);
};
```

student.cpp — розрахунки

```
double Student::avg() const
{
    double sum = 0.0;
    for (int i = 0; i < GRADE_COUNT; i++) sum += grades[i];
    return sum / GRADE_COUNT;
}

bool Student::hasDebt() const
{
    for (int i = 0; i < GRADE_COUNT; i++)
        if (grades[i] < 60) return true;
    return false;
}

bool Student::isExcellent() const
{
    for (int i = 0; i < GRADE_COUNT; i++)
        if (grades[i] < 90) return false;
    return true;
}
```

student_group.cpp — статистика

```
double StudentGroup::groupAvg() const
{
    if (students.empty()) return 0.0;
    double sum = 0.0;
    for (const auto& s : students) sum += s.avg();
    return sum / students.size();
}

double StudentGroup::successRate() const
{
    if (students.empty()) return 0.0;
    int ok = 0;
    for (const auto& s : students)
        if (!s.hasDebt()) ok++;
    return 100.0 * ok / students.size();
}

void StudentGroup::sortByAvg(bool ascending)
{
    std::sort(students.begin(), students.end(),
        [ascending](const Student& a, const Student& b)
        { return ascending ? a.avg() < b.avg() : a.avg() > b.avg(); });
}
```

student_group.cpp — файл

```
bool StudentGroup::saveToFile(const std::wstring& path) const
{
    std::wofstream f(path.c_str(), std::ios::out | std::ios::trunc);
    if (!f.is_open()) return false;
    f.imbue(std::locale(f.getloc(), new std::codecvt_utf8_utf16<wchar_t>));
    f << L'\xFEFF';
    f << L"Name;BookNo;G1;G2;G3;G4;G5\n";
    for (const auto& s : students)
        f << s.toCSV() << L'\n';
    return true;
}

bool StudentGroup::loadFromFile(const std::wstring& path)
{
    std::wifstream f(path.c_str());
    if (!f.is_open()) return false;
    f.imbue(std::locale(f.getloc(), new std::codecvt_utf8_utf16<wchar_t>));
    students.clear();
    std::wstring line;
    bool first = true;
    while (std::getline(f, line))
    {
        if (first)
```

```
        {
            first = false;
            if (!line.empty() && line[0] == L'\xFEFF') line.erase(0, 1);
            if (line.find(L"Name") == 0) continue;
        }
        if (line.empty()) continue;
        Student s;
        if (s.fromCSV(line)) students.push_back(s);
    }
    return true;
}
```

main.cpp — наповнення ListView

```
static void RefreshList()
{
    ListView_DeleteAllItems(g_hList);
    wchar_t buf[32];
    for (size_t i = 0; i < g_group.size(); i++)
    {
        const Student* s = g_group.at(i);
        // ... вставка рядка: №, ПІБ, залікова, 5 оцінок, сер.бал, статус
        const wchar_t* status = s->isExcellent() ? L"Відмінник" :
                                (s->hasDebt()   ? L"Борги" : L"OK");
        ListView_SetItemText(g_hList, (int)i, 9, const_cast<LPWSTR>(status));
    }
}
```

dialogs.cpp — діалог редагування

```
BOOL StudentEditDialog(HWND hOwner, const wchar_t* title, Student& out)
{
    EditCtx ctx{ title, &out, FALSE };
    DialogBoxParamW(GetModuleHandleW(nullptr), MAKEINTRESOURCEW(IDD_STUDENT),
                    hOwner, StudentDlgProc, reinterpret_cast<LPARAM>(&ctx));
    return ctx.ok;
}
```

5. Демонстрація

Рис. 1 — Головне вікно: список групи з сер.балом і статусом

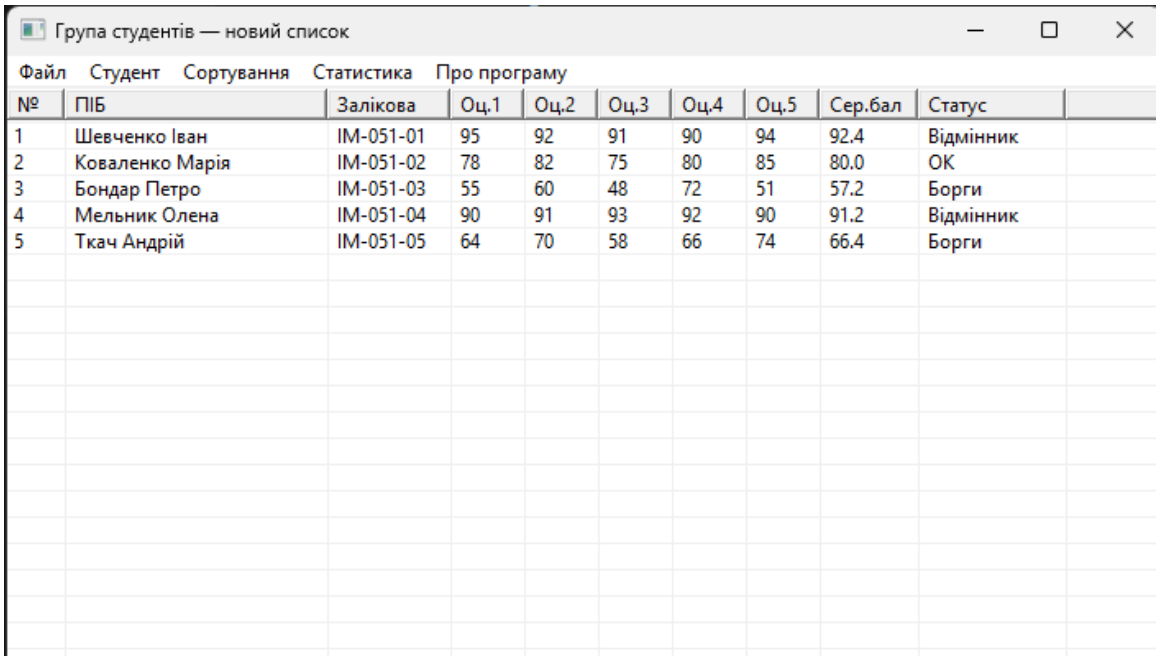


Figure 1: main

Рис. 2 — Діалог додавання студента

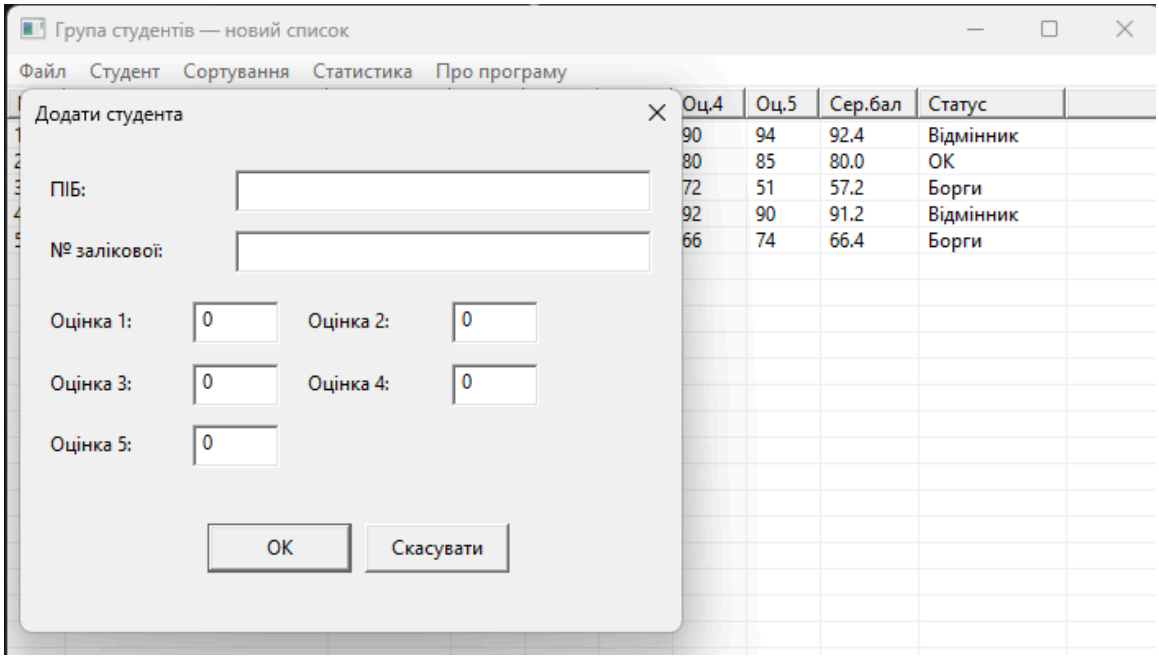


Figure 2: add

Рис. 3 — Діалог статистики групи

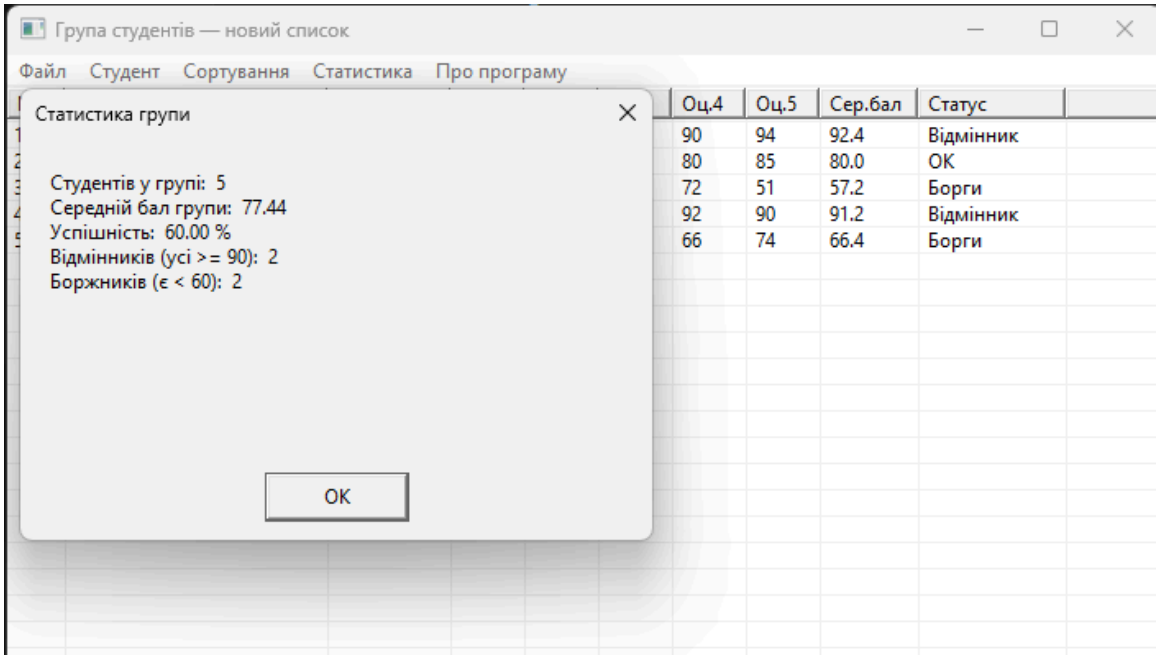


Figure 3: stats

6. Висновки

Розроблено програму ведення списку студентів групи з повним набором розрахунків успішності. Архітектура «модель + UI»: класи `Student`/`StudentGroup` не залежать від інтерфейсу, що дозволяє перевикористати їх у консольній версії. Реалізовано додавання/редагування/видалення, сортування за балом і ПІБ, статистику (середній бал, % успішності, відмінники, боржники), збереження/завантаження CSV-файлу з підтримкою кирилиці.